一、规则

循环移位

- **循环移位的对象：无符号数**
  - 将无符号数二进制形式中的各个位向左或向右移动，被移出的位会重新出现在另一端，形成循环。
- **在很多处理器架构中，循环移位指令会影响状态寄存器中的进位标志CF（Carry Flag）位，CF标志位用于标识在执行算术或逻辑操作时是否发生了进位。**
- **根据CF标志位是否加入循环移位过程，循环移位可分为以下四种：**
  - 不带CF标志位的循环右移
  - 不带CF标志位的循环左移
  - 小循环
  - 带CF标志位的循环右移
  - 带CF标志位的循环左移
  - 大循环

核心规则：循环移位移出的位不丢弃，而是补到另一端的空位，数据位一位不丢，整体相当于首尾相接的环

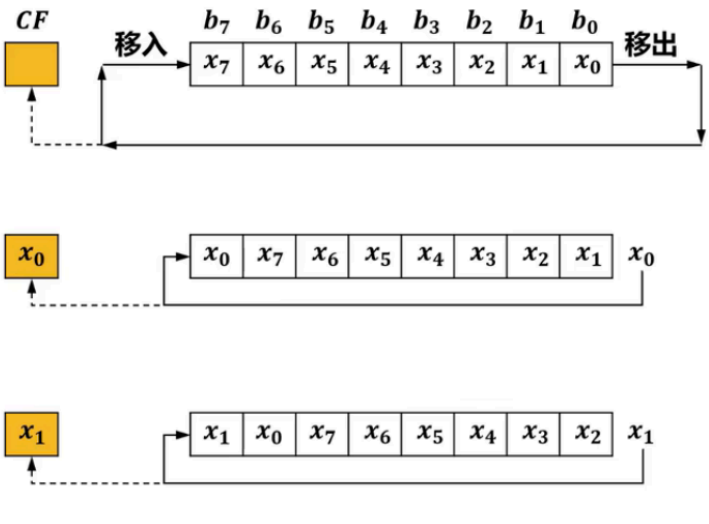
降维类比：像转盘桌上的菜，转到哪都在桌面上，转满一圈回到原位；而逻辑/算术移位像传送带，掉下去的东西（移出的位）再也找不回来

二、分类

注：「不带 CF」指 CF 不参与循环环，但执行后 CF 仍会被移出的位更新；「带 CF」指 CF 本身是环上的一节，相当于 (w+1) 位循环

2.1 不带CF标志位的循环右移

不带CF标志位的循环右移

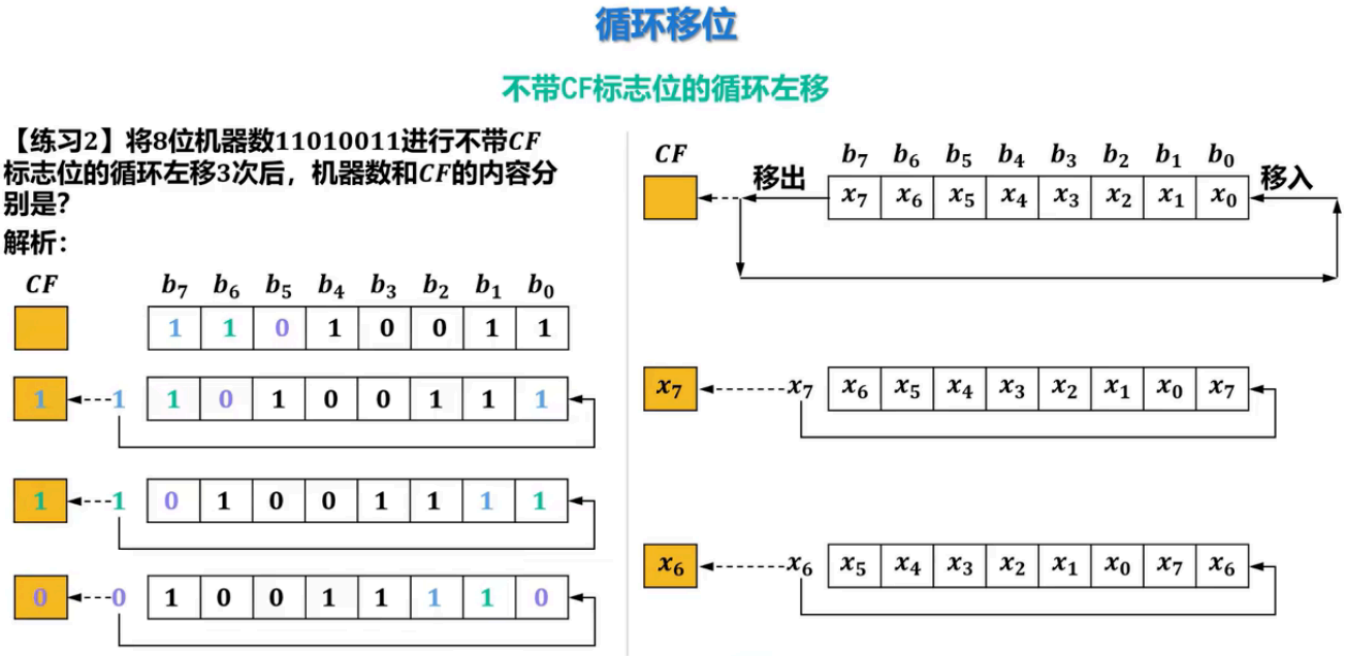


【练习1】将8位机器数11010011进行不带CF标志位的循环右移3次后，机器数和CF的内容分别是？

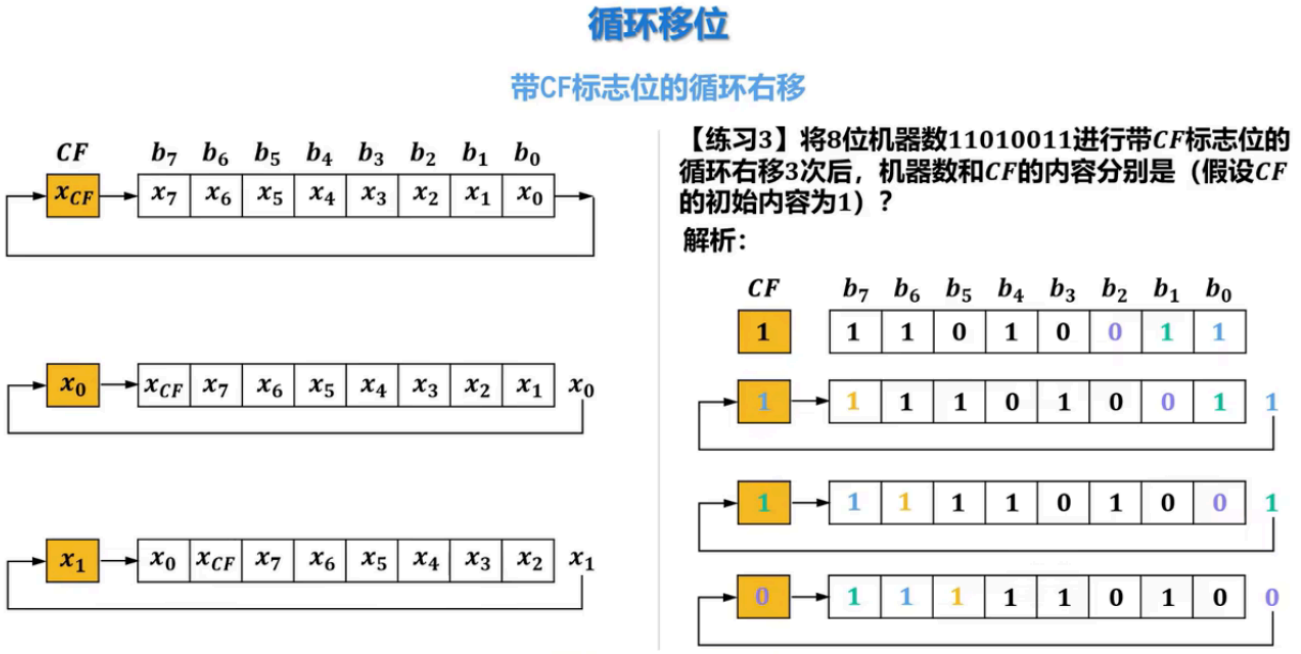
解析：

CF	b_7	b_6	b_5	b_4	b_3	b_2	b_1	b_0
	1	1	0	1	0	0	1	1
1	1	1	1	0	1	0	0	1
1	1	1	1	1	0	1	0	0
0	0	1	1	1	1	0	1	0

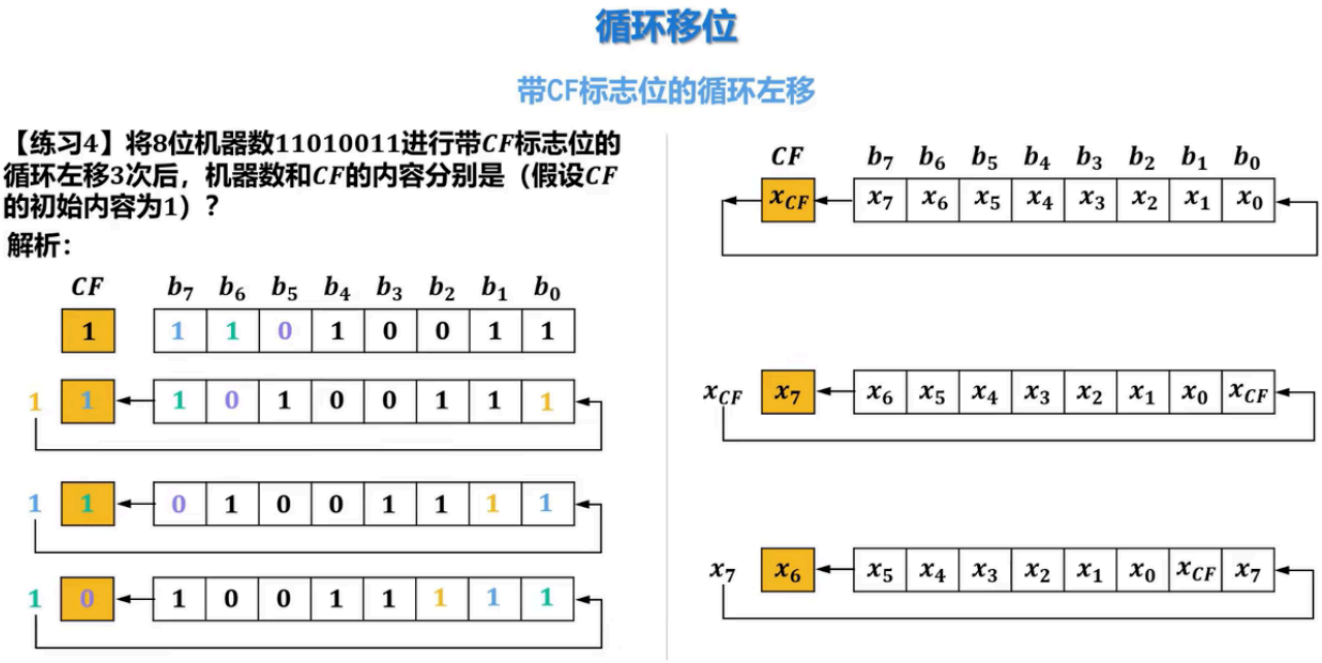
2.2 不带CF标志位的循环左移



2.3 带CF标志位的循环右移



2.4 带CF标志位的循环左移



三、循环移位的应用

3.1 高低字节互换（大小端转换）

大端与小端存储的数据互转，本质就是高字节与低字节对调——这正是 16 位数据循环移位 8 位的效果，也是王道教材所说"数据格式转换"的典型场景

例：`0x1234` 循环左移 8 位 → `0x3412`

```
uint16_t bswap16(uint16_t x) {
    return (x << 8) | (x >> 8);  // 高字节移到低位，低字节移到高位
}
```

3.2 多字长数据的整体移位（带 CF 循环移位的核心用途）

问题：32 位以上的大数要拆到多个寄存器中存储，移位时位如何在寄存器之间"流动"

解法（接力棒类比）：CF 像接力棒——低位寄存器移出的位先递进 CF，高位寄存器再用带 CF 循环移位把 CF 接进自己的低位端

例：R2 存低 16 位、R1 存高 16 位，实现 32 位数整体左移 1 位：

1. R2 逻辑左移 1 位，移出的最高位进入 CF
2. R1 带 CF 循环左移 1 位（RCL），CF 旧值补入 R1 最低位

> **易错**：R2 必须用逻辑左移（把位送出去），R1 必须用带 CF 循环左移（把位接进来）。若 R2 也用不带 CF 的循环左移，最高位会绕回 R2 自己的最低位，位就"串错门"了

3.3 逐位检测且不破坏原数

循环移位转满一整圈后数据恢复原值，适合逐位检查后原样归还的场景

```
int cnt = 0;
uint32_t x = 0x80000001;
for (int i = 0; i < 32; i++) {
    if (x & 1) cnt++;                 // 检查最低位是否为 1
    x = (x << 1) | (x >> 31);         // 循环左移 1 位
}                                     // 32 次后 x 恢复原值
```

四、C语言中的循环移位

C 语言没有原生的循环移位运算符——<<、>> 移出的位直接丢弃，需要手动把丢掉的位拼回来

核心公式（w 位无符号数循环左移 n 位）：`(x << n) | (x >> (w-n))`

```
uint32_t rotl(uint32_t x, unsigned n) {
    n %= 32;                // 移位量规约到 0~31，防止 x >> 32 这类未定义行为
    if (n == 0) return x;  // 特判：n 规约后为 0 时，x >> 32 仍是未定义行为
    return (x << n) | (x >> (32 - n));
    // 左移丢掉的 n 位高位，由右移 (32-n) 位补到低位，实现首尾相接
}
```

循环右移与之对称：

```
uint32_t rotr(uint32_t x, unsigned n) {
    n %= 32;
    if (n == 0) return x;
    return (x >> n) | (x << (32 - n));  // 右移丢掉的低位由左移补到高位
}
```

四个易错点：

1. **x 必须是无符号数**：若 x 有符号，`x >> (32-n)` 是算术右移，负数高位会补符号位 1，污染拼接结果
2. **移位量必须规约**：C 标准规定移位量 ≥ 位宽或为负均为未定义行为，`x >> 32` 在 x86 上实际等价于 `x >> 0`，换平台结果就变
3. **左移溢出**：有符号数 `x << n` 超出表示范围是未定义行为，故循环移位一律用无符号类型实现
4. **循环移位特有等价关系**：循环左移 n 位 ≡ 循环右移 (w-n) 位，逻辑/算术移位没有这种对称性，408 选择题常考

五、易混淆点速查

5.1 带 CF 与不带 CF

对比项	不带 CF 循环移位	带 CF 循环移位
CF 是否在环上	不在环上，但 CF 会被移出的位更新	在环上，构成 (w+1) 位大环
移出位的去向	补到另一端，同时复制一份进 CF	进入 CF，CF 旧值补到另一端空位
典型用途	单寄存器内位重排、字节交换	多寄存器串联的大数整体移位

5.2 循环移位 vs 逻辑/算术移位

对比项	逻辑/算术移位	循环移位
移出的位	直接丢弃	补到另一端，一位不丢
算术意义	左移×2、右移÷2	位重排，无乘除意义
溢出	算术左移会溢出	无溢出概念，位都在
主要应用	算术运算、位掩码	格式转换、大数移位、逐位检测